Guatemala 06 de Octubre de 2026
Dulce Ambrosio – 231143
Daniel Chet – 231177
Cristian Túnchez - 231359

Computación Paralela y Distribuida
Laboratorio 3

Ejercicios

1. **Investiguen qué es AES (Advanced Encryption Standard)**
   AES es un algoritmo de cifrado simétrico por bloques, la misma clave se usa para cifrar y para descifrar. Es una variante del algoritmo Rijndael (Joan Daemen y Vincent Rijmen) y fue adoptado por el NIST como estándar federal en 2001 (FIPS 197), en reemplazo de DES. Opera siempre sobre bloques de 128 bits (16 bytes) y admite claves de 128, 192 o 256 bits, que determinan 10, 12 o 14 rondas respectivamente. La seguridad depende de que la clave sea aleatoria, secreta y de que se use un modo de operación adecuado.

   a) **Identifiquen al menos 3 campos de aplicación.**
   - *Comunicaciones seguras en Internet*: TLS/HTTPS (suites AES-128-GCM y AES-256-GCM), VPN con IPsec y SSH.
   - *Redes inalámbricas:* WPA2 y WPA3 cifran el tráfico Wi-Fi con AES (CCMP/GCMP).
   - *Cifrado de almacenamiento:* BitLocker, FileVault, LUKS/dm-crypt, cifrado de bases de datos y de almacenamiento en la nube (AES-XTS/AES-256).
   - *Mensajería y aplicaciones:* Signal, WhatsApp y gestores de contraseñas protegen mensajes y bóvedas con AES.
   - *Pagos y sector financiero:* protección de datos de tarjetas (PCI DSS), HSM y tokenización, los procesadores modernos aceleran AES por hardware (AES-NI).

   b) **Describan los pasos necesarios para cifrar y descifrar un texto con AES-128. Expliquen:**
   Cifrado de un bloque:
   - *Expansión de clave (KeyExpansion):* la clave de 16 bytes se expande a 11 llaves de ronda de 16 bytes (K0...K10, 44 palabras de 32 bits) mediante RotWord, SubWord y las constantes Rcon.
   - *Ronda inicial:* AddRoundKey, es decir, estado XOR K0.
   - *Rondas 1 a 9:* SubBytes, ShiftRows, MixColumns y AddRoundKey con la llave de la ronda.

- *Ronda 10 (final):* SubBytes, ShiftRows y AddRoundKey con K10, no incluye MixColumns.
- El estado resultante es el bloque cifrado.

Descifrado de un bloque: se ejecutan las operaciones inversas, con las llaves de ronda en orden inverso:

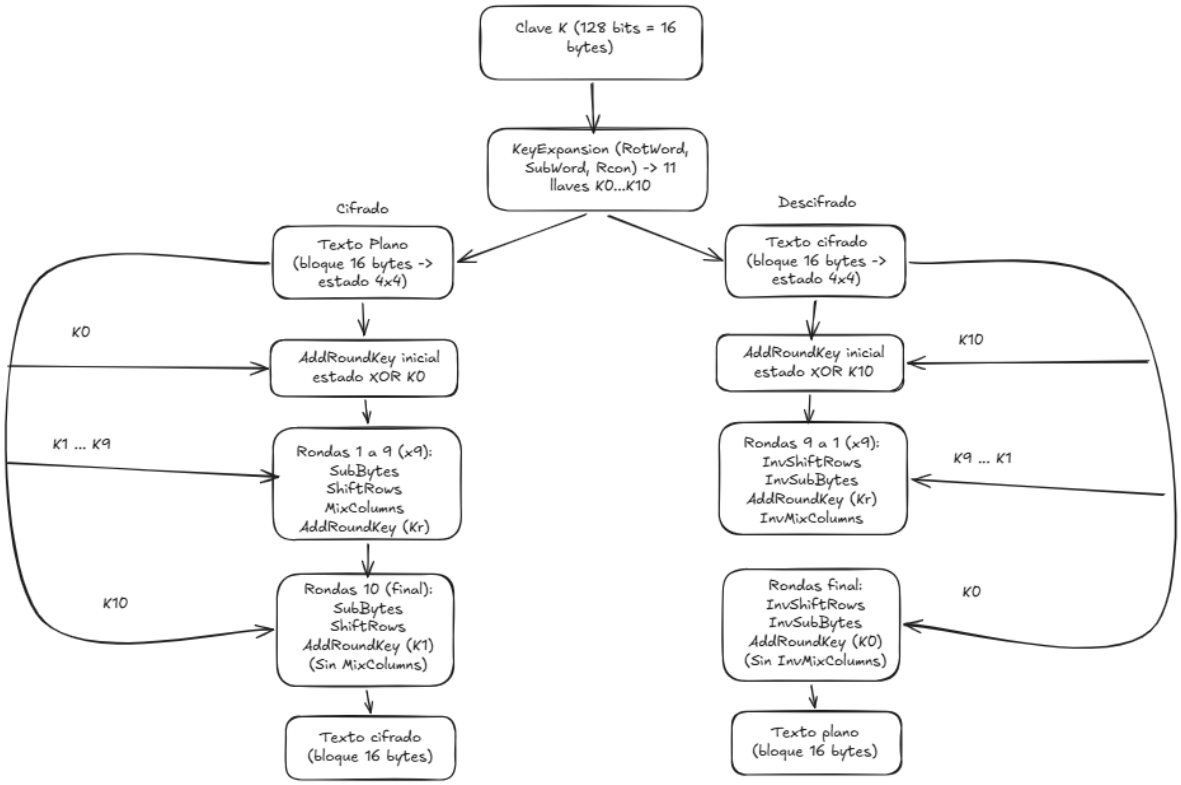
- AddRoundKey inicial con K10.
- *Rondas 9 a 1:* InvShiftRows, InvSubBytes, AddRoundKey (Kr) e InvMixColumns.
- *Ronda final:* InvShiftRows, InvSubBytes y AddRoundKey con K0 (sin InvMixColumns)

- **Cuál es el tamaño de la clave y el procesamiento en bloques**
  La clave mide 128 bits (16 bytes) y el texto se procesa en bloques de 128 bits (16 bytes). Cada bloque se organiza como una matriz de estado de 4x4 bytes (por columnas). Si el mensaje no es múltiplo de 16 bytes se agrega relleno y para más de un bloque, se usa un modo de operación, AES-128 aplica 10 rondas.

- **Las principales transformaciones del algoritmo**

Transformación	Qué hace	Propiedad Criptográfica
SubBytes	Sustituye cada byte mediante la S-box	Confusión (no linealidad)
ShiftRows	Rota la fila i del estado i posiciones a la izquierda (fila 0 no se mueve).	Difusión entre columnas
MixColumns	Multiplica cada columna por una matriz fija sobre GF(2^8)	Difusión dentro de la columna
AddRoundKey	XOR del estado con la llave de ronda correspondiente	Introduce la clave, es su propia inversa
KeyExpansion	Deriva las 11 llaves de ronda a partir de la clave de 128 bits.	La clave interviene en cada ronda

Elaboren un diagrama de flujo: que represente las etapas de cifrado y descifrado de AES-128 y muestren cómo interviene la clave en ambos procesos.



2. **Analicen el programa secuencial: busqueda_clave_aes_secuencial.c, determinen si utiliza correctamente AES-128. Contrasten su implementación con lo investigado sobre el tamaño de la clave, el procesamiento por bloques y el cifrado y descifrado.**

   Si utiliza correctamente AES-128. El programa usa la API EVP de OpenSSL con EVP_aes_128_ecb().

Aspecto	AES-128 (teoría)	Programa Original	¿Cumple?
Tamaño de clave	128 bits = 16 bytes	unsigned char key[16]; EVP_aes_128_ecb() lee exactamente 16 bytes	Sí
Procesamiento por bloques	Bloques de 128 bits (16 bytes)	MESSAGE_LEN = 16 y un _Static_assert garantizan un único bloque	Sí
Relleno (padding)	Necesario si el texto no es múltiplo de 16 bytes	EVP_CIPHER_CTX_set_padding(ctx, 0): correcto porque el mensaje mide exactamente 16 bytes	Sí
Cifrado	Rondas con K0...K10	crypt_block(..., encrypt=1) cifra con SECRET_KEY y obtiene el texto cifrado	Sí

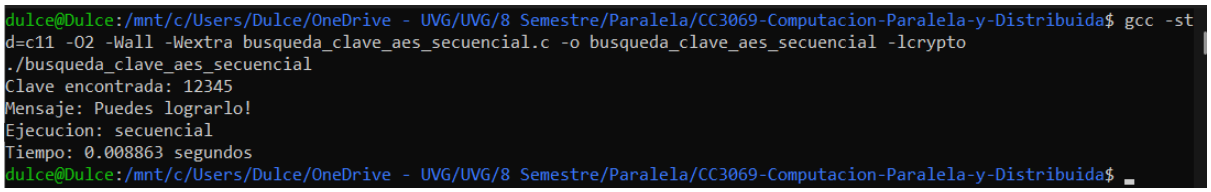
Descifrado	Operaciones inversas con las llaves en orden inverso	crypt_block(..., encrypt=0) descifra con cada candidata y compara con el original	Sí
Clave de 128 bits aleatoria	128 bits de entropía	Solo cambian 20 bits (los otros 108 bits son fijos)	No
Modo de operación	Se recomienda uno con IV/nonce (CBC, CTR, GCM)	ECB, sin IV	No

Las llamadas a AES-128 son correctas (clave de 16 bytes, bloque de 16 bytes, cifrado y descifrado simétricos y sin relleno). Los problemas están en cómo se construye la clave, en el modo ECB y en cómo se verifica cada candidata, como se detalla en el ejercicio 3.

Compilación, ejecución y tiempo de ejecución

El programa ya mide el tiempo con clock_gettime(CLOCK_MONOTONIC) y lo imprime en pantalla (solo la fase de búsqueda, después de preparar el mensaje). Se compiló con los comandos del enunciado, sin ninguna advertencia. Con la clave secreta original (12345) el programa termina casi de inmediato, porque la clave se encuentra después de apenas 12,346 intentos de los 1,048,576 posibles; por eso se midió además una copia del programa con SECRET_KEY = 1048575 (la última candidata, peor caso, obtenida con sed y sin tocar nada más del código).

Figura 1. *Compilación y ejecución del programa original (clave secreta 12345)*



Ejercicio 3. Errores conceptuales y limitaciones

a) **Presten atención especialmente a los siguientes aspectos: la construcción de la clave, el modo ECB, el uso del texto original para verificar las keys candidatas y la diferencia entre el espacio completo de claves de AES-128 y el rango explorado por el programa.**

make_key rellena los 16 bytes con ceros y solo escribe la candidata de 64 bits en los 8 bytes bajos. La clave resultante es no aleatoria y predecible: los 8 bytes superiores siempre son 0x00 (un patrón trivial que cualquier atacante

probaría primero) y, como el rango explorado es de solo 2^{20}, solo 20 de los 128 bits varían. Eso equivale a una clave de 20 bits de entropía, no de 128: el programa no ejemplifica la fortaleza real de AES-128, solo la mecánica de la fuerza bruta

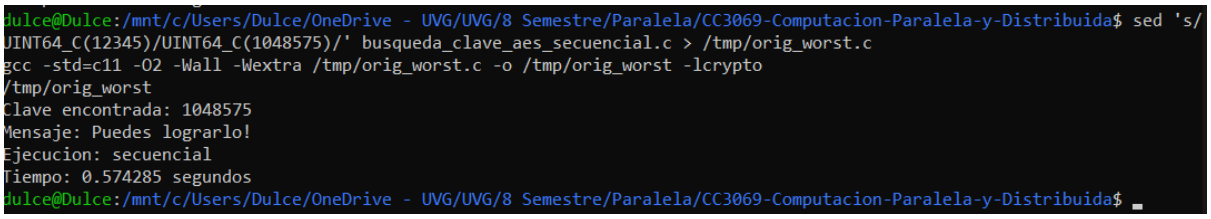
ECB cifra cada bloque de forma independiente y determinista, sin IV ni nonce: bloques de texto iguales producen bloques cifrados iguales (se filtran patrones del mensaje), no hay difusión entre bloques, no autentica los datos y permite reordenar o reutilizar bloques. Aquí no se nota porque se cifra un solo bloque, pero es un modo que no debe usarse en la práctica; lo correcto es CBC/CTR con IV aleatorio o un modo autenticado como GCM.

El programa compara los 16 bytes descifrados contra message, la misma variable con la que se preparó el cifrado. Es decir, el atacante conoce el texto plano completo, solo es realista como ataque de texto plano conocido (en cuyo caso bastaría cifrar ese texto con cada candidata y comparar). En un ataque real normalmente se conoce únicamente un fragmento (un encabezado, un formato) o se usan heurísticas. Además, el mensaje queda duplicado en el binario, lo que rompe la separación entre quien cifra y quien ataca.
El espacio completo de claves de AES-128 es de $2^{128} \approx 3.4\,x\,10^{38}$ candidatas, mientras que el programa solo explora $2^{20} = 1{,}048{,}576$, es decir, una fracción de 2^{-108} ($\approx 3 \times 10^{-33}$) del espacio total. Con la velocidad medida del programa original (≈ 1.8 millones de claves por segundo, a partir de los 0.574 s del peor caso) recorrer todo el espacio tomaría unos 5.9×10^{25} años. El ejercicio solo es factible porque el rango se reduce artificialmente a 20 bits y el secreto se coloca dentro de ese rango; no demuestra que AES-128 sea débil.

Otras limitaciones:

- El secreto original (12345) está al inicio del rango, por lo que la ejecución dura milisegundos y no es representativa para medir rendimiento (por eso se midió también el peor caso, con la clave 1048575).
- El rango y la clave secreta están fijos en el código.
- En cada candidata se vuelve a obtener el algoritmo y se reinicializa todo el contexto de OpenSSL, lo que hace más lento el programa.
- Las claves candidatas quedan en memoria sin borrarse.
- El programa devuelve EXIT_SUCCESS aunque no encuentre la clave.

Figura 2. *Programa original en el peor caso (clave secreta 1048575)*



b) **Propongan e implementen mejoras: justifiquen cada mejora propuesta, relacionadas a seguridad, flexibilidad o rendimiento del programa. Coloquen el nombre del integrante que propuso cada mejora.**

#	Mejora	Tipo	Justificación	Propuso
1	Rango y clave secreta por línea de comandos (-b bits,-k secreto), con validación	Flexibilidad	Permite colocar el secreto al final del rango (peor caso) para medir tiempos y Speedup con sentido, y probar casos límite y el caso “no encontrada” sin recompilar.	Dulce Ambrosio
2	AES-128-CBC con IV aleatorio (RAND_bytes) en lugar de ECB	Seguridad	Elimina el determinismo de ECB: el mismo texto con la misma clave produce cifrados distintos. El IV no es secreto y viaja con el texto cifrado.	Dulce Ambrosio
3	Verificación con un fragmento conocido (crib “Puedes”) y resto imprimible, sin usar el mensaje completo	Seguridad / realismo	El atacante ya no necesita el texto original completo. Probabilidad de falso $positivo \approx 2^{-48}\left(\frac{95}{256}\right)^{10}$ por candidata, despreciable en 2^20 intentos.	Daniel Chet
4	Prefijo de clave no nulo (8 bytes fijos y conocidos) y borrado seguro de la clave en memoria (OPENSSL_cleanse)	Seguridad	Evita la clave trivial de ceros en los bytes altos y no deja material de clave en la pila. La parte fija se declara explícitamente como conocida por el atacante.	Daniel Chet
5	Cifrado cargado una vez (EVP_CIPHER_fetch), contexto reutilizado y solo se cambia la	Rendimiento	Evita repetir trabajo invariante en cada candidata (pedir el algoritmo y reinicializar todo el contexto). En el peor caso el	Cristian Túnchez

	clave en cada intento; se omite Final (sin relleno)		tiempo baja de 0.574 s a 0.110 s (unas 5.2 veces más rápido) con el mismo rango.	
6	Código de salida distinto de cero si no se encuentra la clave; se informa cuántas candidatas se probaron	Corrección	Permite que scripts detecten el fallo y sirve para comprobar que no se omiten candidatas.	Cristian Túnchez

Ejercicio 4. Desarrollen una versión paralela con Open MPI

a) **Desarrollen una versión paralela a partir del programa secuencial corregido y mejorado. Distribuyan la búsqueda entre los procesos sin omitir ni repetir candidatas y coordinen su finalización cuando se encuentre la clave o se agote el rango.**

La versión paralela divide las 2^{20} candidatas en 256 bloques de 4096 y los reparte cíclicamente entre los procesos. El proceso 0 genera el IV, cifra el mensaje y los difunde con MPI_Bcast. Al final de cada ronda todos ejecutan MPI_Allreduce(MPI_MIN). Así todos se detienen en la misma ronda, sin bloqueos, y el resultado es la primera clave válida, igual que en la versión secuencial. El proceso que halla la clave envía el mensaje al proceso 0 para imprimirlo. El tiempo se mide con MPI_Barrier y MPI_Wtime, y se reporta el máximo entre procesos. También se suman las candidatas probadas por todos para comprobar la cobertura.

Verificación: Se probaron las claves 0, 12345, 500000 y 1048575 con la versión secuencial mejorada y con la versión MPI de 4 procesos; en todos los casos ambas recuperan la misma clave y el mismo mensaje ("Puedes lograrlo!"). Si la clave no existe en el rango, la búsqueda termina al agotarse las rondas (ceil(256/n) rondas con n procesos) y la suma de candidatas probadas por todos los procesos es exactamente 1,048,576 con 2, 3 y 4 procesos, lo que confirma que no se omiten ni se repiten candidatas.

Figura 3. *Compilación y ejecución del programa secuencial mejorado*

```
dulce@Dulce:/mnt/c/Users/Dulce/OneDrive - UVG/UVG/8 Semestre/Paralela/CC3069-Computacion-Paralela-y-Distribuida$ cd Labo
ratorio3
gcc -std=c11 -O2 -Wall -Wextra busqueda_clave_aes_secuencial_mejorado.c -o busqueda_clave_aes_secuencial_mejorado -lcryp
to
./busqueda_clave_aes_secuencial_mejorado
./busqueda_clave_aes_secuencial_mejorado -k 1048575
Clave encontrada: 12345
Mensaje: Puedes lograrlo!
Ejecucion: secuencial (mejorado)
Rango explorado: 2^20 = 1048576 candidatas
Candidatas probadas: 12346
Tiempo: 0.001617 segundos
Clave encontrada: 1048575
Mensaje: Puedes lograrlo!
Ejecucion: secuencial (mejorado)
Rango explorado: 2^20 = 1048576 candidatas
Candidatas probadas: 1048576
Tiempo: 0.145238 segundos
dulce@Dulce:/mnt/c/Users/Dulce/OneDrive - UVG/UVG/8 Semestre/Paralela/CC3069-Computacion-Paralela-y-Distribuida/Laborato
rio3$
```

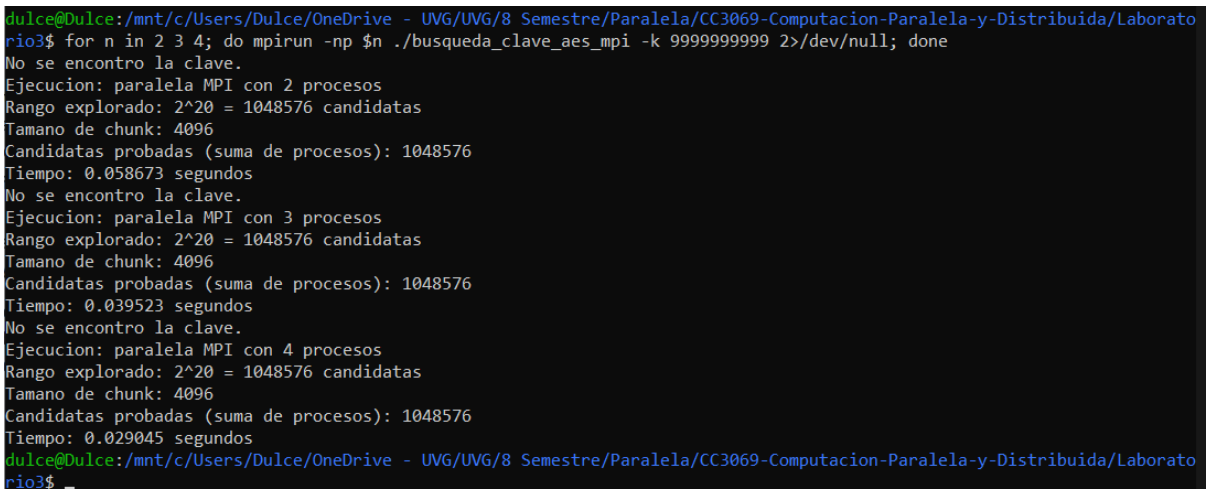
Figura 4. *Compilación y ejecución con 2, 3 y 4 procesos*

```
dulce@Dulce:/mnt/c/Users/Dulce/OneDrive - UVG/UVG/8 Semestre/Paralela/CC3069-Computacion-Paralela-y-Distribuida/Laborato
rio3$ mpicc -std=c11 -O2 -Wall -Wextra busqueda_clave_aes_mpi.c -o busqueda_clave_aes_mpi -lcrypto
mpirun -np 2 ./busqueda_clave_aes_mpi -k 1048575
mpirun -np 3 ./busqueda_clave_aes_mpi -k 1048575
mpirun -np 4 ./busqueda_clave_aes_mpi -k 1048575
Clave encontrada: 1048575
Mensaje: Puedes lograrlo!
Ejecucion: paralela MPI con 2 procesos
Rango explorado: 2^20 = 1048576 candidatas
Tamano de chunk: 4096
Candidatas probadas (suma de procesos): 1048576
Tiempo: 0.073697 segundos
Clave encontrada: 1048575
Mensaje: Puedes lograrlo!
Ejecucion: paralela MPI con 3 procesos
Rango explorado: 2^20 = 1048576 candidatas
Tamano de chunk: 4096
Candidatas probadas (suma de procesos): 1048576
Tiempo: 0.052728 segundos
Clave encontrada: 1048575
Mensaje: Puedes lograrlo!
Ejecucion: paralela MPI con 4 procesos
Rango explorado: 2^20 = 1048576 candidatas
Tamano de chunk: 4096
Candidatas probadas (suma de procesos): 1048576
Tiempo: 0.037157 segundos
dulce@Dulce:/mnt/c/Users/Dulce/OneDrive - UVG/UVG/8 Semestre/Paralela/CC3069-Computacion-Paralela-y-Distribuida/Laborato
rio3$
```

Figura 5. *Comparación secuencial vs. MPI (4 procesos)*

```
dulce@Dulce:/mnt/c/Users/Dulce/OneDrive - UVG/UVG/8 Semestre/Paralela/CC3069-Computacion-Paralela-y-Distribuida/Laborato
rio3$ for k in 0 12345 500000 1048575; do
  ./busqueda_clave_aes_secuencial_mejorado -k $k | head -2
  mpirun -np 4 ./busqueda_clave_aes_mpi -k $k | head -2
done
Clave encontrada: 0
Mensaje: Puedes lograrlo!
Clave encontrada: 0
Mensaje: Puedes lograrlo!
Clave encontrada: 12345
Mensaje: Puedes lograrlo!
Clave encontrada: 12345
Mensaje: Puedes lograrlo!
Clave encontrada: 500000
Mensaje: Puedes lograrlo!
Clave encontrada: 500000
Mensaje: Puedes lograrlo!
Clave encontrada: 1048575
Mensaje: Puedes lograrlo!
Clave encontrada: 1048575
Mensaje: Puedes lograrlo!
dulce@Dulce:/mnt/c/Users/Dulce/OneDrive - UVG/UVG/8 Semestre/Paralela/CC3069-Computacion-Paralela-y-Distribuida/Laborato
rio3$
```

Figura 6. *Cobertura del rango cuando la clave no existe (sin omitir ni repetir candidatas)*



b) **Midan el tiempo de ejecución con 2, 3 y 4 procesos. Y calculen el Speedup para cada ejecución:**

Los tiempos se midieron en el peor caso (clave secreta 1048575, se recorre todo el rango). Cada valor de la tabla es el promedio de 5 corridas. El Speedup se calcula como S(n) = T_sec / T_par(n), donde T_sec = 0.1100 s es el tiempo medio del programa secuencial mejorado, y la eficiencia como E(n) = S(n) / n.

Cantidad de procesos - n	Tiempo medio (s)	Desv. est. (s)	Speedup calculado	Eficiencia
1 (secuencial mejorado)	0.1100	0.0015	1.00	100 %
2	0.0587	0.0024	1.87	94 %
3	0.0390	0.0008	2.82	94 %
4	0.0303	0.0024	3.63	91 %

El Speedup crece casi linealmente con el número de procesos (1.87, 2.82 y 3.63 para 2, 3 y 4 procesos), con eficiencias de 94 %, 94 % y 91 %. El problema es "embarrassingly parallel": las candidatas son independientes y la única comunicación es un MPI_Allreduce pequeño al final de cada ronda. No se alcanza el Speedup ideal por la sincronización al final de cada ronda, el pequeño desbalance de la última ronda, la parte no paralela (preparación, MPI_Bcast y reducción final) y porque WSL2 comparte CPU con Windows.

Figura 7. *Tiempos de 5 corridas con MPI (4, 3 y 2 procesos)*

```
dulce@Dulce:/mnt/c/Users/Dulce/OneDrive - UVG/UVG/8 Semestre/Paralela/CC3069-Computacion-Paralela-y-Distribuida/Laboratorio3$ for i in 1 2 3 4 5; do mpirun -np 4 ./busqueda_clave_aes_mpi -k 1048575 | grep Tiempo; done
Tiempo: 0.034615 segundos
Tiempo: 0.029028 segundos
Tiempo: 0.029713 segundos
Tiempo: 0.028932 segundos
Tiempo: 0.029212 segundos
dulce@Dulce:/mnt/c/Users/Dulce/OneDrive - UVG/UVG/8 Semestre/Paralela/CC3069-Computacion-Paralela-y-Distribuida/Laboratorio3$ for i in 1 2 3 4 5; do mpirun -np 3 ./busqueda_clave_aes_mpi -k 1048575 | grep Tiempo; done
Tiempo: 0.038756 segundos
Tiempo: 0.040022 segundos
Tiempo: 0.038993 segundos
Tiempo: 0.039339 segundos
Tiempo: 0.037908 segundos
dulce@Dulce:/mnt/c/Users/Dulce/OneDrive - UVG/UVG/8 Semestre/Paralela/CC3069-Computacion-Paralela-y-Distribuida/Laboratorio3$ for i in 1 2 3 4 5; do mpirun -np 2 ./busqueda_clave_aes_mpi -k 1048575 | grep Tiempo; done
Tiempo: 0.056114 segundos
Tiempo: 0.057255 segundos
Tiempo: 0.061004 segundos
Tiempo: 0.061596 segundos
Tiempo: 0.057722 segundos
dulce@Dulce:/mnt/c/Users/Dulce/OneDrive - UVG/UVG/8 Semestre/Paralela/CC3069-Computacion-Paralela-y-Distribuida/Laboratorio3$
```

Figura 8. *Tiempos de 5 corridas del secuencial mejorado*

```
dulce@Dulce:/mnt/c/Users/Dulce/OneDrive - UVG/UVG/8 Semestre/Paralela/CC3069-Computacion-Paralela-y-Distribuida/Laboratorio3$ for i in 1 2 3 4 5; do ./busqueda_clave_aes_secuencial_mejorado -k 1048575 | grep Tiempo; done
Tiempo: 0.107476 segundos
Tiempo: 0.110747 segundos
Tiempo: 0.110760 segundos
Tiempo: 0.110145 segundos
Tiempo: 0.111075 segundos
dulce@Dulce:/mnt/c/Users/Dulce/OneDrive - UVG/UVG/8 Semestre/Paralela/CC3069-Computacion-Paralela-y-Distribuida/Laboratorio3$
```

LINK DEL REPOSITORIO:

La solución implementada se encuentra en el siguiente repositorio (clic en la imagen):

